How Far Up Should the C++ Go?

A poker trainer with every layer above the GPU written by hand: text input, focus, scrolling, layout, theming. Convention writes those layers in JavaScript, over elements the browser already supplies. Here, 85.7 percent of the source does work the browser would have done, and rewriting it the conventional way would delete only 37 percent of that.

Dao Huynh · University of California, Los Angeles · daohuynh@g.ucla.edu

Introduction



The trainer freezes one poker hand and starts a clock. Facing a bet, the player types the pot odds, their outs, the win chance those outs convert to, and the average dollars calling gains or loses. Betting instead, they type the fold rate a bluff needs to break even, then pick the best of four bet sizes. Every answer is graded against an exact value, inside a fixed margin.

The code that computes and grades those answers is 1,240 lines; the application is 35,824. Lines of source measure one thing here, maintained surface: code somebody has to read, test, port, debug, and explain to whoever inherits it.

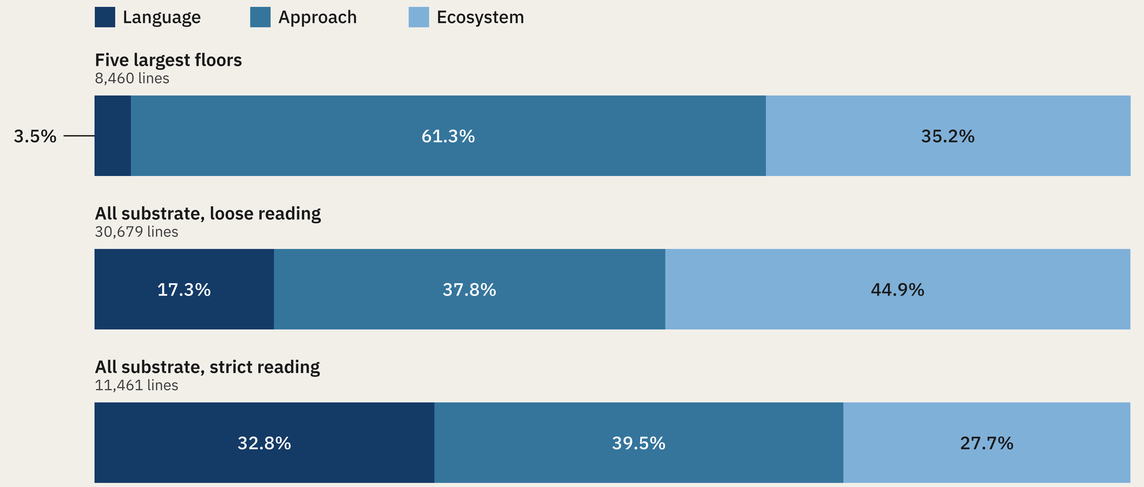
Method

Every file in the source tree goes into exactly one of three categories, decided by three questions asked in a fixed order. The first one answered yes wins. **Domain:** does this file compute or grade the poker decision itself? **Platform:** is its main job something the browser would have done if the project had asked? That category is what this poster calls **substrate**. **Inherent:** everything left over, work any target would have required.

Question two reads two ways. *What did I write that the browser would otherwise have supplied?* is the ownership question. *What does the browser supply outright, doing the whole job?* is the recovery question. All 246 files were classified under both.

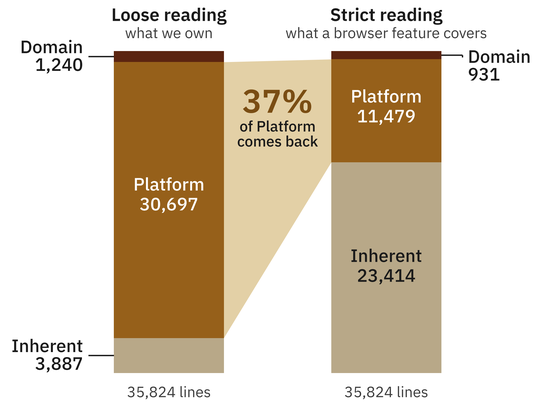
Features are held equal, not implementations. A hand-built dropdown and the browser's own `select` element are the same feature.

Figure 1 Which switch pays



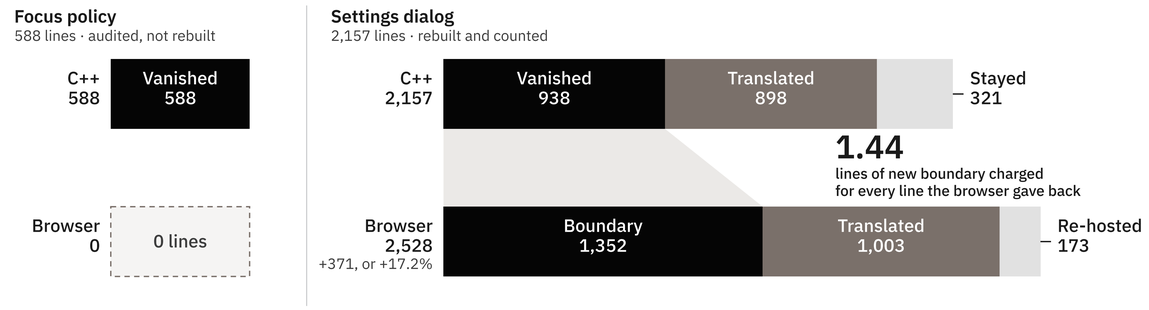
Each line of substrate was assigned to the single cause whose removal alone would delete the most of it. Approach means immediate mode: nothing on screen persists between frames, so the application holds state a conventional toolkit keeps inside the widget. Approach leads on the five largest floors and under the strict reading. Language is never higher than second.

Figure 2 What comes back



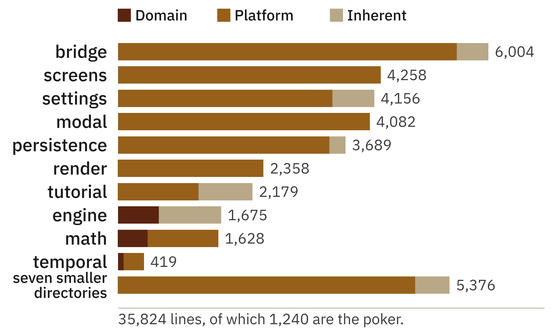
Both columns classify the same 35,824 lines. The loose reading asks what we wrote that the browser would have supplied. The strict reading asks what a browser feature covers outright. Only the Platform band answers whether a rewrite would delete the code: 30,697 lines down to 11,479, so 37 percent comes back. The rest does not vanish. It moves into work any target would have required.

Figure 3 Both sides priced



A browser supplies every one of the focus system's sixteen keyboard behaviours, so all 588 lines go. The settings dialog was rebuilt at fixed feature parity and came out 17.2 percent larger. 938 lines of focus and keyboard machinery genuinely vanished, and 1,352 lines of boundary arrived to replace them: a payload, a diff, a dispatch and a handshake where a function call used to be. The measured population is 2,157 of that floor's 3,478 lines. Most of the boundary is paid once, so a second browser dialog would cost about 19 percent less. The first one is the one that looks worst.

Figure 4 The bill



Ten directories named, seven collapsed. Bar length is total lines on one scale. All 1,240 poker lines sit in named bars: engine 665, math 482, temporal 93. Three bars read 100 percent Platform, the classifier over-collecting.

Takeaways

Every project picks a stack, and most pick by habit or by what the author already knows. That is reasonable, and it is how most software gets built. It is also a decision with a price, and the price is not usually counted. This one was.

Asked which part of the program held the poker mathematics, I named the wrong one, six weeks after writing it. Then this poster estimated that a conventional rewrite would delete over half the source; it deletes under a third. Both errors flattered the argument, and counting caught both.

This measures what a project would get back by rewriting a layer the conventional way. It does not say which layers should move.

Limitations

- One codebase, no control. The method may not be the cause.
- Counts are raw lines; `cloc` returns 21,972, not 35,824.
- Directory-level classification puts `render` and `screens` at 100 percent.
- Seams are invisible to a line count; 37 percent is a ceiling.
- Domain admits only the arithmetic, not the cards or the table.
- Time per zone and defect density were attempted; both failed.
- 378 lines of hand-written JavaScript count here as C++.
- The second-dialog saving is a projection, not a measurement.
- Four files counted as source are largely data tables.
- The settings floor includes 749 lines of a different dialog.

Supporting material

live app

github

linkedin